

Paradigma funcional:
Haskell {

Lugo Aguilar Damian Alexander,
23490386

Guevara Quiñonez Luis Alonso
23490377

}



Introducción {

Haskell es un lenguaje de programación puramente funcional, cuya primera versión fue lanzada en 1990. Su nombre proviene del matemático Haskell Brooks Curry, que sentó las bases de los lenguajes de programación funcional con su trabajo sobre lógica combinatoria

Haskell se basa en el cálculo lambda (lenguaje formal para la investigación de funciones), por lo que el logotipo del lenguaje contiene el símbolo de esta letra griega.



Filosofia {

La filosofía del lenguaje Haskell se fundamenta en la creación de software robusto, mantenible y matemáticamente elegante a través de la programación funcional pura.

Un lenguaje funcional es paradigma basado en el uso de funciones matemáticas puras, evitando el estado mutable y los datos compartidos. Se enfoca en el "qué" a diferencia del declarativa que se enfoca en el "cómo" hacerlo

Tradicional

```
bool estaCosme = false;
foreach (var alumno in alumnos)
{
    if (alumno.Name == "Cosme")
    {
        estaCosme = true;
        break;
    }
}
```

Funcional

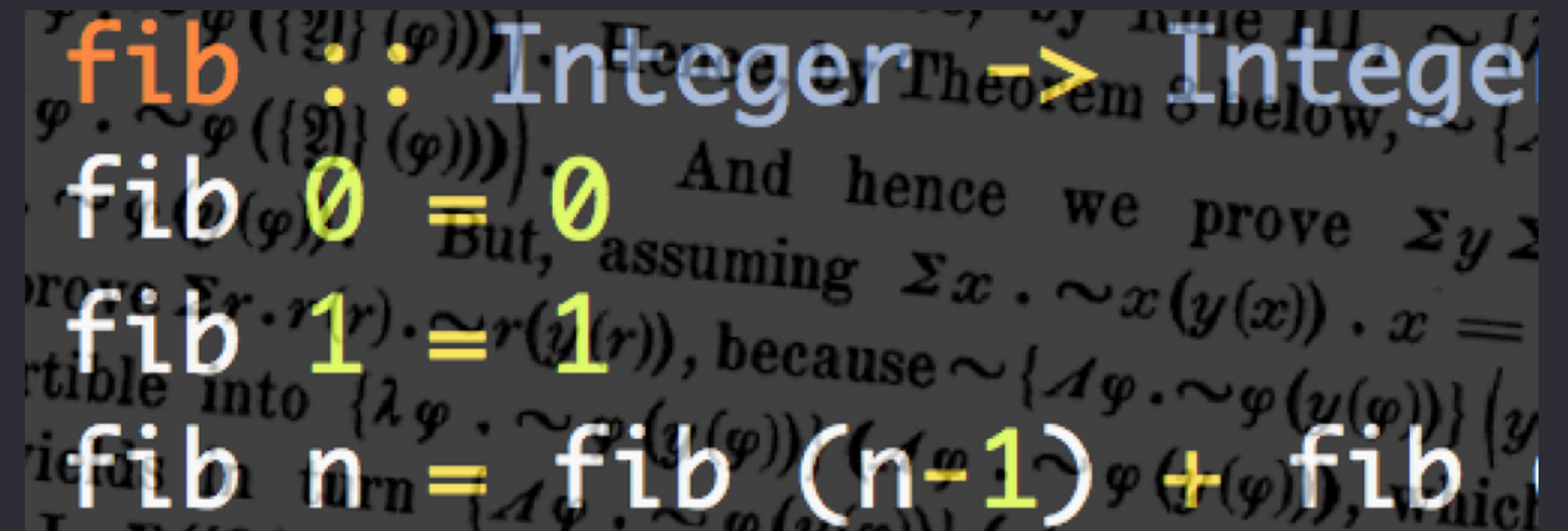
```
alumnos.Any(
    a => a.Name == "Cosme"
```

}

Evaluación de expresiones {

Haskell se caracteriza por usar una evaluación de expresiones llamada "Lazy evaluation" (evaluación floja/perezosa)

esto significa que las expresiones no se calculan hasta que sus valores son estrictamente necesarios para el resultado final. Esto permite mayor eficiencia, evitar cálculos innecesarios y manejar estructuras de datos infinitas.



The image shows a snippet of Haskell code for a Fibonacci function, overlaid on a background of mathematical text. The code is as follows:

```
fib :: Integer -> Integer
fib 0 = 0
fib 1 = 1
fib n = fib (n-1) + fib (n-2)
```

The background text includes phrases like "Theorem 3 below", "And hence we prove", "But, assuming", and "because", along with various mathematical symbols and expressions.

}

Evaluación de expresiones {

01

Una expresion
no se evalua a
menos que sea
necesaria

02

Mejora el
rendimiento

03

Puede definir
estructuras
infinitas

04

Evita
calculos
innecesarios

}

Tipos de datos {

Los tipos de datos de haskell son bastante parecidos a otros lenguajes, ya sea tipos basicos como :

- **int**: Enteros de tamaño fijo
- **char**: Un solo carácter Unicode, delimitado por comillas simples
- **bool**: Valor booleano: True o False

Tipos compuestos como:

- **listas**: Secuencias homogéneas (elementos del mismo tipo) de tamaño variable
- **tuplas**: Secuencias heterogéneas (pueden contener diferentes tipos) de tamaño fijo.
- **funciones**

o tipos algebraicos como:

- **OR**: Sumas
- **AND**: Productos

Sin embargo este lenguaje tambien permite definir nuevos tipos de datos, esto se hace de la siguiente manera:

- **data**: Permite definir tipos de datos algebraicos (sumas y productos), por ejemplo: `data Persona = Persona String Int`
- **type**: Crea sinónimos de tipos, útil para hacer alias más legibles (ej. `type Nombre = String`)
- **newtype**: Crea un tipo nuevo a partir de uno existente con restricciones, más eficiente que data para envolver un solo tipo.

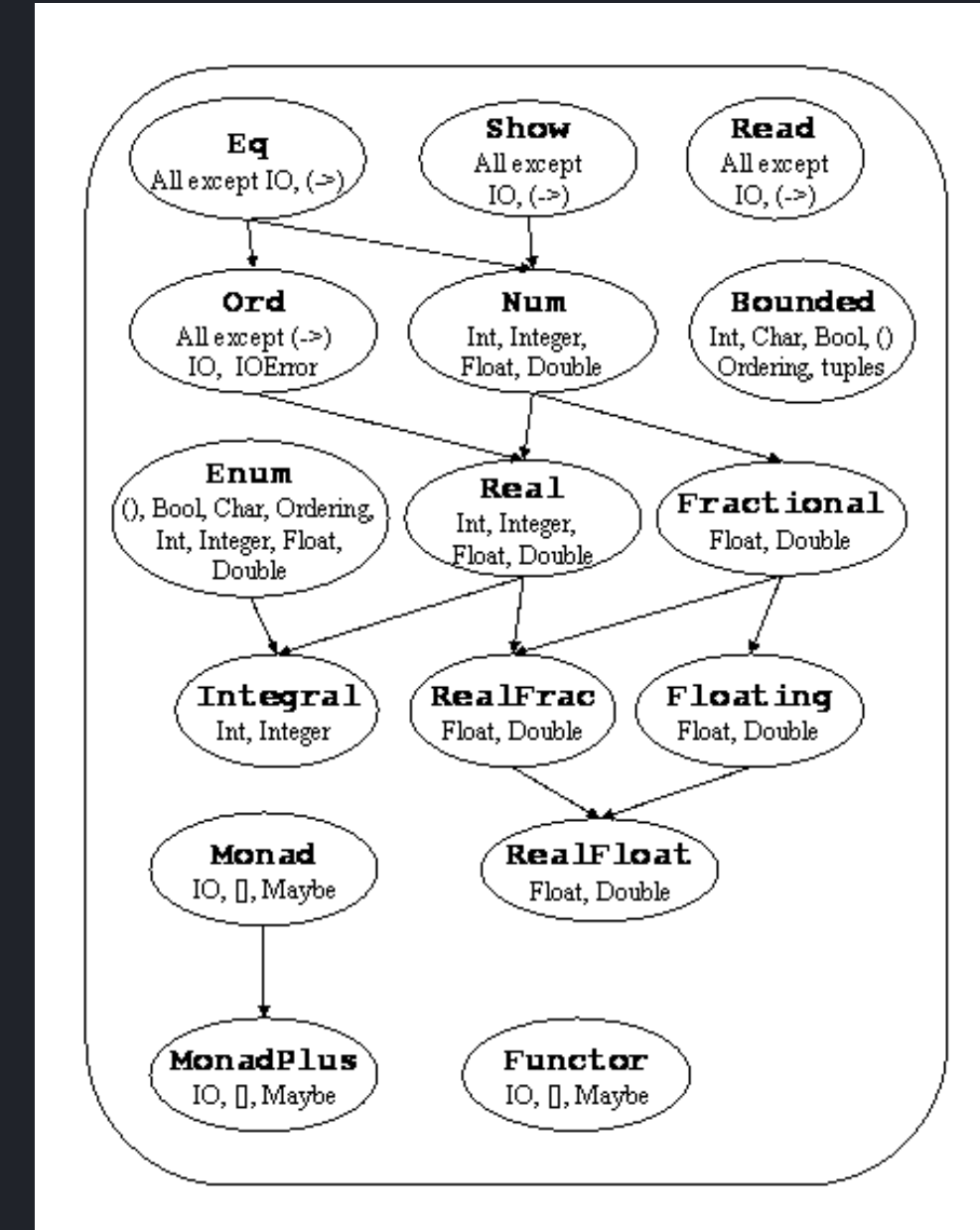
}

Disciplina de datos {

La disciplina de datos es conjunto de reglas que definen cómo se clasifican, almacenan y utilizan los datos en un lenguaje, en este caso haskell cuenta con un sistema de tipos estático, fuerte y con inferencia automática

Esto quiere decir que su disciplina de tipos

- los tipos se verifican en compilación
- No hay conversiones implícitas peligrosas
- No siempre necesitas declarar tipos



}

Funciones o estructuras clave {

Como ya se menciono anteriormente haskell es un lenguaje de programación puramente funcional, famoso por su evaluación diferida (o lazy evaluation), esta tambien tiene con algunas funciones como las que se mencionaron antes (tuplas y listas) y otras estructuras clave que se mencionaran a continuacion:

- **Pattern Matching**: Coincidencia de patrones, fundamental para descomponer estructuras de datos como listas (x:xs) o definir casos base y recursivos.
- **Curricación (Currying)**: Todas las funciones en Haskell técnicamente aceptan un solo argumento y devuelven una nueva función si se necesitan más argumentos.
- **Composición de Funciones (.)**: Permite unir funciones (f . g) para mayor legibilidad y modularidad.



}

Ventajas {

1. Seguridad de tipos

Sus estándares de programación funcional y su enfoque teórico se traducen en un sistema estático mucho más sólido y robusto, capaz de detectar y eliminar errores de compilación antes de ejecutar el programa, además permite crear propiedades súper específicas para después verificarlas en tiempo de compilación

- Sistemas Financieros.
- Criptografía.
- Compiladores y Análisis de Datos.

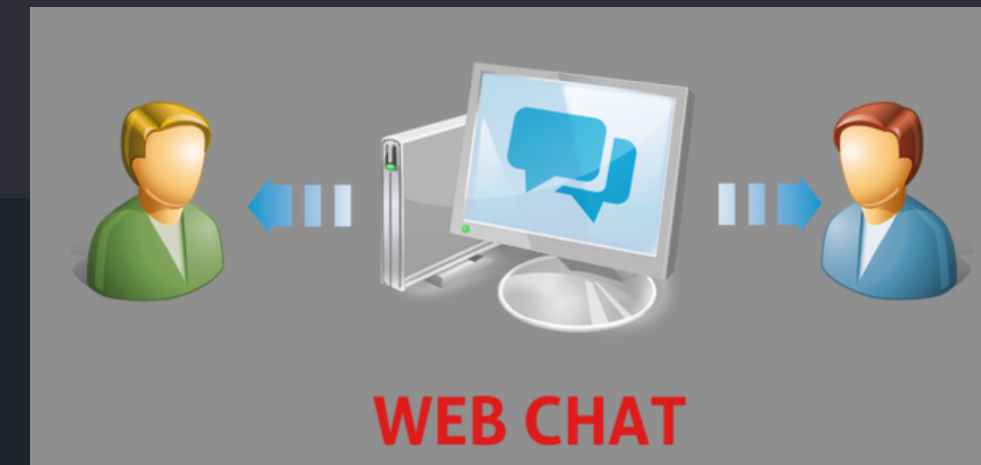


2.-Concurrencia y paralelismos controlados

Perfecto para evitar problemas comunes inherentes a la concurrencia, además, cuenta con principios, herramientas y abstracciones para controlar las concurrencias.

- Utiliza un modelo de hilos ligeros para la ejecución simultánea de tareas.
- Permite crear miles y millones de hilos concurrentes.
- Puedes optimizar y coordinar la ejecución de los hilos concurrentes y sus respectivas tareas sin sobrecargar el sistema

- Servidores de Chat o Mensajería



}

Ventajas {

3. Componentes reutilizables

Haskell facilita y promueve la creación de abstracciones, también facilita la creación de programas a partir de componentes reutilizables, lo que te permitirá mantener y extender el código a largo plazo.

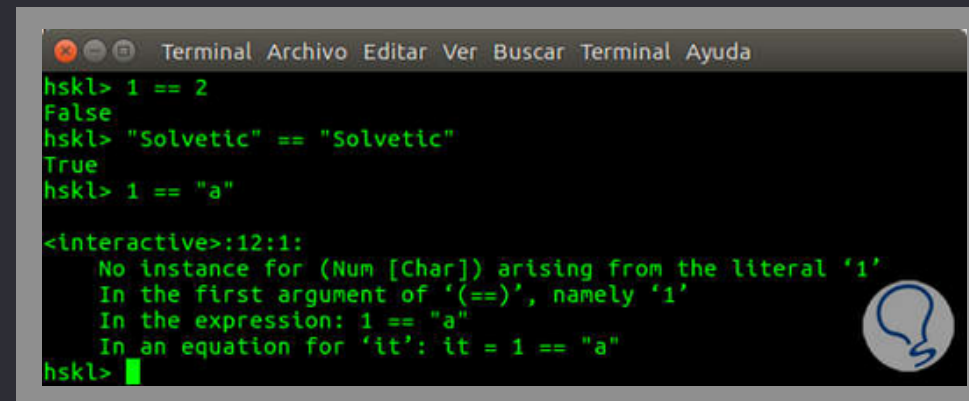
Y como sus componentes son reutilizables, podrás hacer toda clase de modificaciones, experimentos y réplicas sin arriesgar el código base de tu sistema.

Arquitecturas de Microservicios.

4. Ecosistema

Cuenta con un ecosistema repleto de herramientas y utilidades para todo tipo de proyectos.

- GHC (Glasgow Haskell Compiler): su compilador principal con múltiples extensiones de lenguaje y un intérprete GHCi, entre otras funciones.
- Stack: para manejar las dependencias y crear entornos de desarrollo coherentes y fáciles de replicar.
- Cabal: para la construcción y administración de paquetes. También para crear, instalar y gestionar bibliotecas.
- Hackage: por así decirlo, la biblioteca central de Haskell para todo tipo de paquetes y proyectos.



}

Desventajas {

1.-Ausencia de sistema de macros

Esta es una espada de doble filo.

Por un lado, Haskell es un lenguaje inmutable perfecto para prevenir errores. Sin embargo, este modelo no permite la creación de macros, o sea, no permite crear abstracciones para secuencias de código en tiempo de compilación. En otras palabras: las automatizaciones son más difíciles y limitadas.

Lo que sí permite es la automatización a través de plantillas basadas en la metaprogramación. Pero tendrás que familiarizarte primero con su biblioteca de plantillas.

2.Actualizaciones de compilador

Quizá la mayor desventaja de este lenguaje. Las actualizaciones del compilador de Haskell pueden introducir cambios en la sintaxis, las reglas de tipos y el comportamiento del lenguaje.

Como resultado, el código existente puede no ser compatible con las nuevas versiones del compilador, obligándote a modificar código manualmente para que pueda ejecutarse correctamente con la nueva versión.

}

Desventajas {

3. Curva de aprendizaje

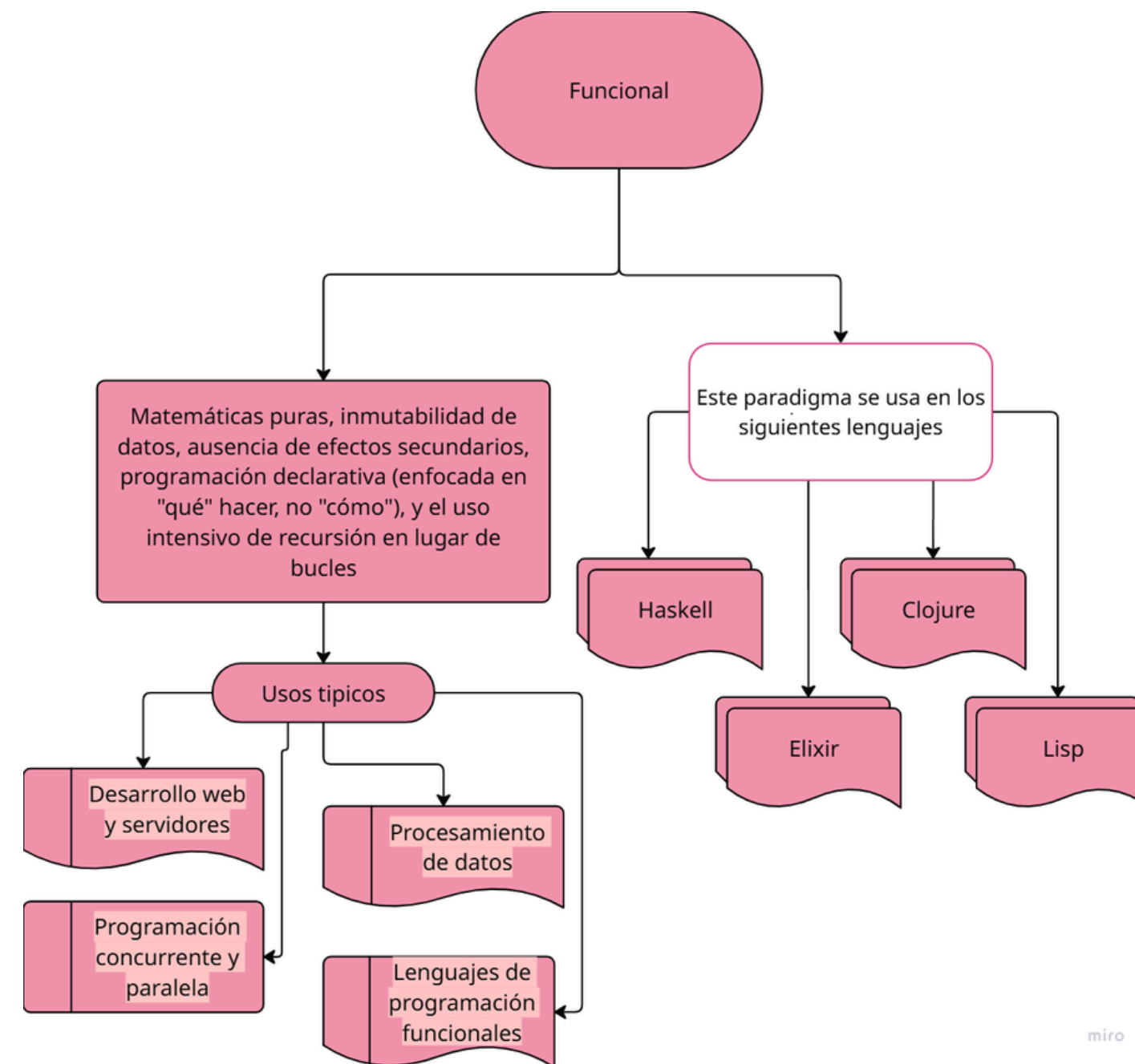
Más que difícil, Haskell es “diferente” a la gran mayoría de los lenguajes de programación más populares.

La dinámica de Haskell, basada en la programación funcional y en la evaluación perezosa, va en contra de la mayoría de los lenguajes de programación más populares.

Entonces, ¿qué tan difícil es aprender Haskell? Haskell no es tan complejo como muchos pueden creer, pero sí es un lenguaje exigente y no se aprende de la noche a la mañana.

}

Mapa Conceptual Visual {



Paradigma	Estilo/ Sintaxis	Que problemas resuelve mejor	Ejemplo
Funcional	El uso de funciones de orden superior, la composición de funciones, la recursión y el procesamiento de flujos de datos (streams).	Problemas que requieren alta concurrencia, procesamiento paralelo, manipulación intensiva de datos y sistemas robustos o previsibles	Criptomonedas, validación de transacciones y consenso en redes distribuidas.
Orientado a objetos	Diseño basado en clases, la programación visual, y el desarrollo basado en componentes	Complejidad en el desarrollo de software de gran escala, facilitando el mantenimiento, la reutilización de código y la modelización de procesos del mundo real	Desarrollo de aplicaciones móviles, juegos, interfaces de usuario y sistemas empresariales.
Lógico	y predicados, en lugar de secuenciar instrucciones	los problemas basados en hechos, reglas, relaciones y deducción lógica	Inteligencia Artificial, ideales para aplicaciones que requieren representar conocimiento, razonamiento simbólico y toma de decisiones basada en reglas

{

!!GRACIAS POR ESCUCHAR¡¡

}

Referencias {

- IONOS. ¿Qué es Haskell? Todo sobre el lenguaje de programación. <https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/que-es-haskell/>
- The Haskell Programming Language. Haskell.org. <https://www.haskell.org/>
- ¡Aprende Haskell por el bien de todos! Tipos y clases de tipos. <http://aprendehaskell.es/content/Tipos.html>
- ¡Aprende Haskell por el bien de todos! La sintaxis de las funciones. <http://aprendehaskell.es/content/Funciones.html>
- Sancho, F. Haskell: el Lenguaje Funcional. https://www.cs.us.es/~fsancho/Blog/posts/Haskell_el_lenguaje_funcional.md
- ThePowerTech. (2023, 29 de agosto). Haskell: descubre el lenguaje de programación funcional del momento. <https://thepowertech.es/blog/haskell-lenguaje-de-programacion>
- Inesdi. Programación funcional. <https://www.inesdi.com/blog/programacion-funcional/>
- Reddit. ELI5: The differences between programming paradigms. Recuperado de https://www.reddit.com/r/explainlikeimfive/comments/1cy86p5/eli5_the_differences_between_programming/?tl=es-419